Real-Time Transit Reliability Lakehouse

Deriving arrival times that no agency publishes, and grading the predictions riders see

Aatish Lobo Borna Karimi

MSDS 682, Data Stream Processing, Summer 2026

Contents

1. Problem, user, and result

1.1 The problem

GTFS-Realtime publishes two live feeds. **TripUpdates** carries predictions ("trip X will reach stop 22 at 15:07"). **VehiclePositions** carries GPS observations. Neither states that a vehicle *arrived* at a stop. That fact must be derived.

Delay is `actual - scheduled`. The scheduled time is published and exact, so every error in the derived actual passes directly into the delay figure, and no downstream step can detect it. The risk is bias, not noise: random error averages out across a route, while a systematic offset shifts an entire distribution invisibly.

1.2 Target user and result

Primary user: a rider deciding whether to trust the estimate in their app. **Secondary:** an analyst comparing operators, or an agency auditing its own predictions.

Result: when an agency says the bus arrives in N minutes, how wrong is it? Reported by lead time, route, and hour of day, with a model that measurably improves those predictions.

1.3 Why the result is credible

The **prediction** is the agency's forecast from TripUpdates. The **actual arrival** is derived by us from VehiclePositions, by observing `STOPPED_AT`. The feeds share no data and no logic.

Deriving arrivals from the last prediction before a vehicle passes (*prediction settlement*) would instead measure the agency against itself and produce a small error that means nothing. See Section 7.2.

2. Data source and classification

2.1 Source

Name	511 SF Bay Open Data, GTFS-Realtime transit feeds
Owner	Metropolitan Transportation Commission (MTC)
Portal	https://511.org/open-data/transit
Format	GTFS-Realtime (Protocol Buffers, proto2)
Coverage	All Bay Area operators, one regional feed (`agency=RG`)
Access	Free API key; 60 requests per 3,600 seconds

Data provided by 511.org (Metropolitan Transportation Commission), http://www.511.org. Full schema and rights: `DATA_SOURCE.md`.

2.2 Classification

Hybrid, with a real-time core. The feeds are polled continuously and streamed through Kafka. The submitted review path is a deterministic replay of archived data, requiring no API key and producing identical results on every run.

2.3 Rate limits and their consequence

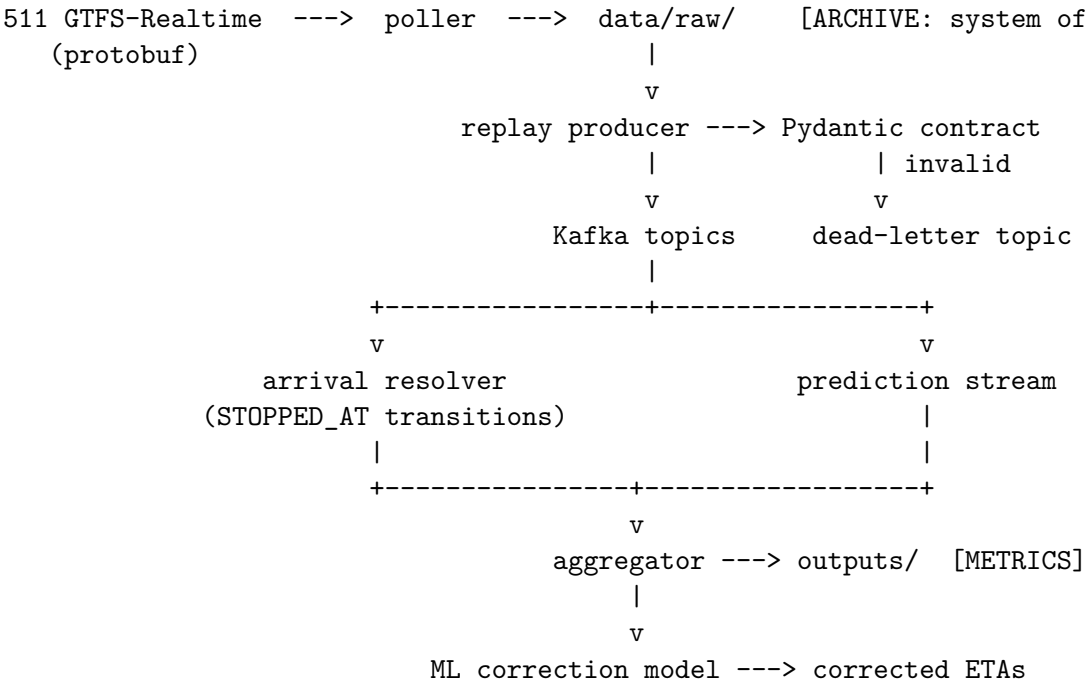
60 requests/hour across two feed types caps polling at one cycle per 120 seconds, exactly at the limit. The budget is enforced client-side by a sliding-window limiter rather than by reacting to HTTP 429, because a throttled token halts archiving and GTFS-Realtime has no history endpoint. Minutes lost are lost permanently.

The poll interval is the noise floor of every derived arrival: an arrival cannot be observed more precisely than the interval that sampled it. It is stored on every row as `poll_interval_s`, so a later change in cadence cannot silently invalidate older data.

2.4 Licensing and repository privacy

The 511 Data Disseminator Agreement grants a broad licence in Section 1, but **Section 2(c)** requires written acceptance from any third party before providing them the data. The committed replay sample is verbatim 511 protobuf, which is the Provided Data itself rather than a derivative work, so a public repository would distribute it with no acceptance secured. The repository is **private**, with access granted individually. Section 5(b) attribution appears in the README, in `DATA_SOURCE.md`, and programmatically in the output artifacts.

3. Architecture



3.1 Two structural decisions

The archive is the system of record; Kafka is transport. Topics carry 24-hour retention deliberately. GTFS-Realtime cannot be re-fetched, so anything not on disk is lost permanently, and a topic treated as durable storage silently expires with no recovery path.

Live and replay share one code path. The poller only writes to disk; everything downstream

only reads from disk. Live and replay differ solely in whether new files keep appearing, so a reviewer exercises the real pipeline rather than a separate test utility that would drift from it.

3.2 Ingest

The poller fetches both feeds every 120 seconds and **writes raw bytes to disk before attempting any decode**. A decoding bug can be repaired by reprocessing the archive; a poll never taken is gone. Decode failures quarantine the payload rather than discarding it.

Also implemented: stale-feed detection by payload hash and header timestamp, single-instance enforcement by lock file, connection retry that charges every attempt to the rate budget, and redaction of the API key from all logs.

3.3 Streaming

Four topics, created explicitly rather than auto-created:

Topic	Partitions	Retention	Purpose
`gtfsrt.trip_updates.v1`	3	24 h	agency predictions
`gtfsrt.vehicle_positions.v1`	3	24 h	GPS observations
`gtfsrt.dead_letter.v1`	1	7 d	records failing validation
`transit.arrival_events.v1`	3	7 d	derived arrivals

Partition count was chosen, not defaulted. Assignment is computed from the key hash, so raising the count later changes which partition a key maps to, splitting a key's history and breaking ordering retroactively for data already written.

Cleanup policy is `delete`, never `compact`. Compaction retains only the latest message per key. These topics are an event log, and the resolver works by reading state transitions across consecutive observations, so compaction would destroy exactly what it consumes.

4. The event contract

Pydantic v2 models in `streaming/contracts.py`. Every record is validated before publication; failures route to a dead-letter topic rather than crashing the consumer or being dropped.

4.1 Why a contract exists

1. **It marks a boundary of ownership.** Once a record is on a topic, any consumer may read it. The contract is the only statement of what they may assume.
2. **It detects drift.** A field the contract does not declare raises immediately under `extra="forbid"`.
3. **It makes bad records routable rather than fatal.**

4.2 The invariant the contract must not break

Pydantic **v2** is a hard requirement. Version 1 coerced `None` to a field's declared default. Since 44.2% of records carry no delay value (Section 6.1), a default of zero would have relabelled roughly

54,000 records per poll as exactly on time, undoing the founding invariant at the one boundary built to protect it. Strict mode is enabled for the same reason: a wrong-typed value signals upstream change and must fail rather than be repaired.

4.3 Partition key

```
key = f"{service_date}:{trip_id}"
```

Kafka guarantees ordering only within a partition and routes by key hash. Keying on the trip puts every observation of one trip in one partition, in order, which the resolver requires because it detects arrivals from state transitions across consecutive polls.

`service_date` is included because `trip_id` recurs daily. Keying on `trip_id` alone would force different days of the same scheduled trip into one partition, coupling them and skewing load.

5. Deriving arrivals

5.1 Three methods, one implemented

Method	Signal	Weakness
A, position state	`current_status == STOPPED_AT` at a stop	not published by all operators
B, geofence	nearest approach of GPS trail to stop	measures closest approach, biased early
C, prediction settlement	last prediction before passing	inherits agency error, enables leakage

Method A is implemented. Method C is effectively unavailable on this feed (Section 6.1) and would compromise the evaluation. Method B requires GTFS-Static stop coordinates, a separate source outside the scope taken.

5.2 The arrival is the first sighting, not the last

A vehicle waiting at a stop reports `STOPPED_AT` on every poll for the duration:

```
poll 1:  approaching stop 25
poll 2:  STOPPED_AT stop 25     <- the arrival
poll 3:  STOPPED_AT stop 25     <- still waiting
poll 4:  approaching stop 26
```

The last such observation measures departure; an intermediate one measures nothing. Only the first is the arrival. The resolver therefore keeps per-trip state, which is what makes the partition key in Section 4.3 load-bearing.

5.3 Provenance on every row

```
{
  "service_date": "20260806", "trip_id": "SF:12053041_M11",
  "stop_sequence": 47, "stop_id": "13550", "route_id": "SF:1",
```

```
  "actual_arrival_ts": "2026-08-06T21:52:43+00:00",
  "arrival_method": "stopped_at", "arrival_confidence": "high",
  "poll_interval_s": 120, "resolver_version": "1.0.0"
}
```


Resolver availability varies by operator. Storing only the timestamp would let *which agency a vehicle belongs to* silently determine *how accurate its arrival time is*, and later analysis would report agency effects that are artifacts of our own code. Storing the method makes that variation filterable, and `poll_interval_s` travels with each row as its error bar.

Records that produce no arrival are **counted, not discarded**; the counts are the measurement of resolver coverage per operator.

6. Findings from the live feed

Measured via `profiling/profile_feed.py` and the evaluation harness.

6.1 Feed characteristics

28 operators in the regional feed, not the four commonly named.

44.2% of StopTimeUpdate rows carry no delay value, 54,638 of 123,735 in one sample. Nineteen of 28 operators never populate it; AC Transit produced 23,469 consecutive absent rows. Because the format is proto2, a naive read returns `0` for both absent and exactly-on-time. Implemented that way, this project would have reported 54,638 records as perfectly punctual: a fabricated on-time spike, plausible in appearance, corrupting every downstream metric.

Only 15 of 28 operators publish `current_status`, which method A requires. Muni populates it 99.2% of the time; Caltrain, 0%.

Settled predictions are effectively unavailable. Only 4 of 28 operators emit `uncertainty = 0`, at negligible volume. Muni never publishes uncertainty at all, reconfirmed when model training dropped the column as entirely absent.

Seven operators revisit the same `stop_id` **within a trip** (Emery Go-Round: 23 of 29 trips), which is why the grain is `(service_date, trip_id, stop_sequence)`.

A single payload spans multiple service dates. One poll held trips dated 20260805 (58,971 rows), 20260806 (263 rows, past midnight), and 2,858 rows with no date; positions in the same payload reached back eight days. Service date is a property of a row, not a payload, so the raw archive partitions on ingest date.

6.2 Sampling limits

Vehicles dwell for seconds while we sample every 120 seconds. On SF Muni, **90.3% of captured stop events appear in exactly one poll** and capture is about **21%** of stop events. Three distinct effects, which must not be conflated:

1. **Coverage.** Most stop events are missed. A sample-size limit, not an accuracy one.
2. **One-sided offset.** A vehicle is observable as `STOPPED_AT` only between arriving and departing, so a derived timestamp falls at or after true arrival, never before. Derived arrivals are biased **late**.

3. **Selection bias.** Capture probability scales with dwell time, so terminals, layovers, and timepoints are over-represented. Median observed dwell among multi-poll sightings is 353 seconds.

Effects 2 and 3 both push measured arrivals later, making agencies appear more optimistic than they may be. **Every bias figure here is an upper bound.**

7. Results

7.1 Prediction accuracy (SF Muni, 25,501 prediction-arrival pairs)

lead time	n	bias (s)	median abs err (s)	p90 (s)	within 60 s	within 180 s
0-2 min	509	-11	13	52	92.7%	100.0%
2-5 min	4,603	-46	50	112	59.5%	98.4%
5-10 min	4,897	-66	70	172	43.3%	91.2%
10-20 min	8,919	-98	102	257	31.1%	77.0%
20+ min	6,573	-142	146	375	23.0%	59.3%

Short-horizon predictions are excellent: under two minutes out, median error is 13 seconds and 92.7% land within a minute. **Accuracy degrades steadily with horizon:** by 20 minutes, 23% fall within a minute. **Bias is negative at every horizon and grows**, meaning the agency predicts earlier than observed, by 142 seconds on average at long horizons, subject to Section 6.2's upper bound.

Median absolute error describes typical magnitude; **mean signed error is the bias**, and bias is what matters, because random error averages out across a route while systematic optimism does not.

One exclusion: a prediction issued *after* the vehicle arrived is a correction, not a forecast, and scoring it would count hindsight as foresight. Forty pairs excluded, counted rather than silent.

7.2 The bounded AI element

Task. Predict the residual `actual_arrival - agency_predicted_arrival` and apply it as a correction. Predicting zero is identical to trusting the agency, which makes the baseline comparison exact.

Model. `HistGradientBoostingRegressor`, seven features, 1,008,746 training rows, 336,529 test rows, split **strictly forward in time**.

	MAE (s)	RMSE (s)	bias (s)	within 60 s
baseline: trust the agency	195.2	381.9	-154.0	31.0%
baseline: add global mean	215.1	358.2	+78.5	14.8%
baseline: add mean per horizon	206.7	359.2	+74.8	22.4%
model	**166.6**	**305.6**	**+43.7**	**32.1%**

14.7% lower error than the agency's own prediction, beating all three baselines, with per-horizon improvement from 12.5% (20+ min) to 19.9% (5-10 min). Notably, **both naive bias corrections perform worse than trusting the agency**, because residuals are right-skewed (mean +213 s, median +111 s) and adding the mean overcorrects the typical case. Section 10 develops this.

Leakage defences

This is deliberately a **prediction-correction model**, declared rather than obscured. Using an agency forecast as a feature is normally leakage; it is legitimate here because **labels come from VehiclePositions and features from TripUpdates**, so the label cannot contain the feature.

- `lead_time` **is excluded.** Defined as `actual_arrival - issued`, it contains the target and is unknowable until the vehicle has arrived. It would produce a spectacular score and an unusable model. Asserted by test.
- **The split is temporal, not random.** Random splitting leaks twice: one trip contributes many rows that scatter across both sides, and later observations would train a model tested on earlier ones.
- **A feature that improved the score was removed.** Day-of-week cut MAE by 10 seconds (156.8 s versus 166.6 s) but `dow = 0` occurred only in the test window, so 31.2% of test rows carried a value the model had never seen and inherited an adjacent day's correction through bin placement. The gain was an artifact. The reported 166.6 s is the honest figure.
- **Fallback, enforced in code.** The unmodified agency prediction is served when no artifact exists, when the model failed to beat its best baseline at training time, or when input falls outside the trained range. The baseline is the default; the model is an override that must earn its place on every retrain.

8. Evaluation and validation evidence

8.1 Acceptance checks

`evaluation/run_acceptance_checks.py` verifies properties against the data the pipeline actually produced, as distinct from unit tests, which verify functions on imagined inputs.

Check	Result
`null_zero_discrimination`	408,149 rows against raw protobuf, **0 violations**
`contract_validation`	245,701 valid, 0 invalid
`grain_uniqueness`	6,046 arrivals, 6,046 distinct keys
`idempotent_resolution`	two independent runs, bit-for-bit identical
`provenance_complete`	0 missing fields
`event_time_not_ingest_time`	0 collisions with processing time

The first is the strongest artifact: it re-parses the raw protobuf and compares field presence against decoded output rather than trusting the decoder's account of itself. Across 408,149 rows (199,675 absent, 10,693 explicit zeros, 197,781 real values), zero violations.

Every check carries a row-count floor and reports `UNMEASURED` rather than `PASS` on insufficient data. A check that passes on an empty table reports green while measuring nothing, and keeps reporting green after an upstream break empties its input.

A seventh check reports `INFO`, not `PASS`: it found no loop-route revisits in the sample window and therefore asserted nothing. The behaviour it would have tested is asserted directly in the unit tests.

8.2 Unit tests

83 tests covering the absent-versus-zero invariant, contract validation, resolver semantics, aggregation statistics, connection recovery, secret redaction, and 13 tests dedicated to ML leakage defences.

8.3 Pipeline throughput

Stage	Volume	Time
Replay producer	1,668,118 rows, 0 dead-lettered	77 s
Arrival resolver	38,701 positions to 6,046 arrivals	16 s
Full `make demo`	cold start to passing checks	2 min 15 s

9. Review path

Non-cloud, locally runnable, no API key required.

```
make venv && source .venv/bin/activate
make demo
```

Starts Kafka in Docker, creates topics, replays 39 minutes of archived feed data, derives arrivals, produces metrics, and runs the acceptance checks. About 2 minutes 15 seconds from cold.

Expected output ends with `passed=6 failed=0 unmeasured=0 info=1`, with results in `outputs/` and evidence in `evaluation/`.

Sample data: `data/replay_sample/`, 39 polls over a contiguous 39-minute window (2026-08-06, 14:52-15:32), 13 MB of raw protobuf with a manifest. Three properties are deliberate. It is **raw, not decoded**, so the reviewer exercises the decoder. It is **contiguous**, because arrival detection requires consecutive observations. It is **gap-checked**, because a hole produces missed arrivals indistinguishable from an operator that does not report them.

Cleanup: `make kafka-down`. No cloud resources are provisioned.

10. Optional extension: controlled method comparison

The single labelled extension beyond the required minimum.

Four methods for predicting a vehicle's arrival, on **one input, one temporal split, one named metric** (MAE in seconds): three baselines and the model from Section 7.2.

Method	Rule
`baseline_agency`	Trust the published prediction unchanged. Equivalent to a residual of zero.
`baseline_global_bias`	Add one constant, the mean residual over the training split.
`baseline_horizon_bias`	Add the mean residual for the prediction's lead-time bucket. A five-row lookup table.
`model`	`HistGradientBoostingRegressor` predicting the residual from seven features.

All four use the identical split. The baselines derive their constants from **training rows only**; using the full dataset would let the test set inform its own baseline.

10.1 Exact steps

```
make venv && source .venv/bin/activate
make train
```

About 10 seconds. **No API key, no Kafka, no network.** Independent of the `make demo` path in Section 9.

Input: `ml/data/features_sample.csv.gz`, 168,160 rows, committed. **Saved output:** `ml/artifacts/training_report.json` (metrics, split definition, feature decisions) and `ml/artifacts/prediction_correction_model.joblib`. Both are submitted.

10.2 Expected output

Printed to console and written to the `results` block of `training_report.json`:

Method	MAE (s)	RMSE (s)	bias (s)	within 60 s
`baseline_agency`	194.7	380.9	-152.3	30.9%
`baseline_global_bias`	215.1	358.2	+80.1	14.9%
`baseline_horizon_bias`	206.3	358.9	+76.1	22.6%
`model`	**168.5**	**308.9**	**+45.3**	**31.8%**

Which figures appear where. These come from the committed sample and are what a reviewer reproduces. Section 7.2 quotes 166.6 s, from the full 1.34-million-row archive, which is 137 MB and too large to submit. The 1.9-second gap is the cost of shipping a reviewable subset, stated so a reproduced 168.5 s reads as confirmation. The full archive rebuilds with `make features && make train-full` given a 511 key.

10.3 What the comparison shows

The model wins: 168.5 s against 194.7 s, a 13.5% reduction.

The more useful finding is that **both naive bias corrections lose to doing nothing.** Section 7.1 established that agency predictions are systematically optimistic and that the bias grows with lead time; the obvious response is to add that bias back. Applied directly it makes predictions worse, 215.1 s and 206.3 s against 194.7 s for leaving them alone. The training-split residuals show why:

Lead time	Mean residual	Median residual	Rows
0-2 min	+95 s	+55 s	5,820
2-5 min	+104 s	+72 s	8,918
5-10 min	+134 s	+77 s	13,984
10-20 min	+180 s	+113 s	24,079
20 min+	+295 s	+174 s	73,293

The mean exceeds the median in every bucket by a widening margin. Most vehicles are moderately late while a minority are severely late, and the tail drags the mean above the typical case, so adding the mean overcorrects the ordinary vehicle to accommodate the rare one.

The model is not applying a better constant. It learns a correction **conditional** on lead time, hour, route, and stop position, adding 90 seconds in one context and 250 in another where a lookup table must commit to one value.

This comparison is also the verification method for the AI element, and the fallback in Section 7.2 depends on it: the model is served only if it beat its best baseline at training time. The table is the gate, not a report.

11. Limitations and failures

11.1 Limitations

- **Coverage.** About 21% of stop events, limited by the 60 requests/hour quota. Derived arrivals are biased late and over-represent long-dwell stops (Section 6.2), so reported bias is an upper bound.
- **Scope.** One operator (SF Muni), six days, no seasonal, holiday, or weather variation.
- **Model coverage.** 95.8% of training data falls in hours 14-19, because the machine hosting the archiver sleeps overnight. Effectively an afternoon and evening model.
- **No confidence intervals.** Point estimates only.
- **In-memory join.** The aggregator joins across the full replay window in memory, honest for a bounded 39-minute replay but requiring a windowed join with watermarks at production scale.
- **No Schema Registry.** Compatibility rests on Pydantic contracts and a version stamp rather than central enforcement, so a producer could still publish a breaking change.

11.2 Failures encountered

A mislabelled partition column. A directory named `service_dt=` was filled with the UTC poll date, filing every poll between 17:00 and midnight under the following day: seven hours daily, including all of PM peak. Caught by comparing directory names against the dates inside the files. The fix was not "use the right date" but the realisation that a payload spans multiple service dates and cannot be partitioned by service date at all.

A statistical claim that was wrong. The poll interval was first described as "plus or minus 60 seconds of noise". Measuring dwell disproved it: the real effects are a coverage limit, a one-sided late bias, and a selection bias. Three different things, and the correction materially weakens the headline claim.

A silent duplicate-counting bug. Replaying twice doubled every sample count while leaving every mean, median, and percentile identical. The output looked correct while `n` misreported. Caught by tearing down Kafka and re-running clean.

An API key leaked into a log. The HTTP library embeds query parameters in exception messages, so failed fetches wrote the full URL to disk. Caught by a pre-push secret scan; now redacted at source and covered by tests.

A misleading green check. An acceptance check reported `PASS` while asserting nothing, because the sample contained no instances of the case it tested.

The common property: **every one produced output that looked correct**, several producing better-looking numbers than the correct version. None crashed. Reading the code would have caught almost none of them. Running against real data and checking output against independent measurement did.

11.3 Deviations from the proposal

A plain-Python consumer replaces Spark Structured Streaming. The proposal listed this as a sanctioned fallback; it was taken up front rather than discovered late. Delta Lake, the SCD2 schedule dimension, dbt, Dagster, and Kubernetes consequently fall outside the delivered scope.

Prediction accuracy replaces schedule-based on-time performance. True OTP requires GTFS-Static and an as-of join against a slowly-changing dimension. The delivered metric exercises the identical streaming path with no extra source.

12. Next steps

1. **Request a rate-limit increase.** Faster polling attacks coverage, one-sided offset, and selection bias at once. The highest-value improvement available.
2. **GTFS-Static integration** for true on-time performance, via an SCD2 schedule dimension and as-of join.
3. **Additional operators**, beginning with VTA, which showed strong resolver A availability.
4. **Geofence resolver (method B)** for the 13 operators where method A cannot fire, calibrated against A where both are available.
5. **Confidence intervals** on model output.
6. **Windowed join with watermarks**, replacing the in-memory aggregation.

13. Contributions

Two-person team, split along the pipeline: **Borna Karimi owned everything up to Kafka; Aatish Lobo owned Kafka onwards.** The handoff is the validated event contract.

13.1 Borna Karimi, source to contract

Area	Detail
Data source	511 evaluation, API key acquisition, Disseminator Agreement review, the privacy decision in Section 2.4
Ingestion	The poller: 120-second cadence, client-side rate budget, stale-feed detection, archive-before-decode ordering
Decoding	Presence-aware GTFS-Realtime decoding, the `HasField` handling in Section 4.2
Archive layout	`ingest_dt` partitioning, and the finding that a payload spans multiple service dates
Feed profiling	Field-population analysis establishing that resolver A was viable
Documentation	`DATA_SOURCE.md`

13.2 Aatish Lobo, contract to result

Area	Detail
Event contract	Pydantic models, partition key design, dead-letter routing
Kafka	Topic design, partition count, retention and cleanup policy, idempotent producer, manual offset commits
Streaming	Replay producer, arrival resolver (Section 5), aggregator and the join onto the grain
AI element	Feature construction, temporal split, leakage defences, baselines, model, fallback (Sections 7.2 and 10)

Area	Detail
Evaluation	Acceptance-check suite, unit tests, throughput measurement
Documentation	`README.md`, `AI_USAGE.md`, this report

13.3 Shared

Architecture and the correctness invariants in Sections 4 and 5 were designed jointly, before implementation. Both authors reviewed the other's code, and both can explain the complete path: source, contract, Kafka, resolver, aggregation, model, evaluation. AI assistance is disclosed in `AI_USAGE.md`.

14. References and artifacts

Artifact	Location
Source data documentation	`DATA_SOURCE.md`
AI usage disclosure	`AI_USAGE.md`
Onboarding guide	`docs/ONBOARDING.md`
Pitfall register (60 items)	`docs/PITFALLS.md`
Daily build reports	`docs/reports/`
Model training report	`ml/artifacts/training_report.json`
Acceptance check results	`evaluation/acceptance_report.json`
Produced metrics	`outputs/`

Data source: 511 SF Bay Open Data, Metropolitan Transportation Commission. Data provided by 511.org, http://www.511.org